

Proyecto 1 — QR-NET

Curso: Redes de Computadoras

Institución: Instituto Tecnológico de Costa Rica

Profesor: Kevin Moraga

Fecha: Mayo 2026

Grupo:

Estudiante	Carné	Capa principal
Ariana Jiménez	2023201036	Capa 2/3
Ion Dolanescu	2022049034	Capa 1
Jose Mario Jiménez	2023102334	Capa 7

Tabla de Contenidos

- [1. Introducción](#)
 - [2. Ambiente de Desarrollo](#)
 - [3. Estructuras de Datos y Funciones](#)
 - [4. Instrucciones para Ejecutar el Programa](#)
 - [5. Actividades Realizadas por Estudiante](#)
 - [6. Comentarios Finales](#)
 - [7. Conclusiones y Recomendaciones](#)
 - [8. Bibliografía](#)
-

1. Introducción

En la actualidad, el anonimato y la libertad de expresión son temas cada vez más relevantes a nivel mundial. Existen regiones donde la infraestructura de telecomunicaciones es inaccesible o está bajo el control de gobiernos que restringen la comunicación de sus ciudadanos. El Artículo 19 de la Declaración Universal de los Derechos Humanos garantiza la libertad de expresión, pero su ejercicio práctico sigue siendo un desafío técnico en muchos contextos.

QR-NET es un protocolo de red experimental que propone un canal de comunicación alternativo: la luz visible. Dos nodos equipados con una cámara y una pantalla pueden establecer un enlace transmitiendo códigos QR multicolor entre sí. Este canal no requiere infraestructura de red convencional, no expone direcciones IP ni metadatos de enrutamiento al medio subyacente, y puede atravesar sesiones de videoconferencia para operar a distancia geográfica arbitraria.

El proyecto implementa las capas 1, 2/3 y 7 del modelo OSI para el protocolo **QR-NET Rainbow 1**, con las siguientes características:

- **Capa 1 (Física):** Canal óptico de luz visible con códigos QR de hasta 16 colores por módulo, handshake de 3 vías, tramas de máximo 128 bytes con CRC-16, compresión Zstandard y datacasting de archivos.

- **Capa 2/3 (Red):** Red mesh con enrutamiento dinámico distance-vector, directorio de nodos, circuitos virtuales efímeros para comunicación anónima, y soporte para canales Ethernet (TCP) y WiFi (UDP multicast) además del canal óptico.
- **Capa 7 (Aplicación):** Aplicación de microblogging/chat anónimo sobre la red mesh, con mensajes de tipo directo, publicación y respuesta, identificados por seudónimos efímeros.

Capa 7: AnonChatApp / CLI
Capa 3: QRNetNode / Packet
Capa 2: DispositivoLuzAdaptador
Capa 1: Grid64Codec / ColorPalette OpenCV / Cámara / Pantalla

2. Ambiente de Desarrollo

Lenguaje y versión

Componente	Versión
Python	3.10+
Sistema Operativo	Windows 10/11 (desarrollo), Ubuntu 24 (CI)

Entorno de Desarrollo

Herramienta	Uso
VS Code	Editor principal con extensiones Python, Pylance y Python Test Explorer
PyCharm	IDE secundario usado por Ion para el desarrollo de Capa 1
Git	Control de versiones
GitHub	Repositorio remoto y revisión de Pull Requests
pytest	Framework de testing unitario e integración
virtualenv / venv	Entorno virtual Python aislado

Dependencias Python

```
opencv-python # captura de cámara, renderizado de grilla, detección de contornos
numpy         # operaciones matriciales sobre frames e imágenes
zstandard    # compresión Zstandard (zstd) para payload
pillow       # captura de pantalla para test de loopback óptico
screeninfo   # obtener dimensiones del monitor para renderizado fullscreen
```

```
cv2-enumerate-cameras # listado de cámaras disponibles (find_camera.py)
pytest               # ejecución de tests
```

Instalar todas las dependencias con:

```
pip install opencv-python numpy zstandard pillow screeninfo cv2-enumerate-cameras
pytest
```

Estructura del repositorio

```
proyecto-qr/
├── common/                                # Utilidades compartidas
│   ├── checksum.py                       # CRC-16, CRC-32, SHA-128
│   ├── exceptions.py                     # Excepciones del dominio
│   ├── network_policies.py               # Enums, tipos MAC y NodeID
│   └── other.py                           # CompressionAlgorithm y otros
├── transmision/                           # Capa 1 – canal óptico QR
│   ├── frames.py                         # HandshakeFrame y DataFrame
│   ├── adapter.py                        # DispositivoLuzAdaptador
│   ├── interfaces.py                     # Contratos abstractos de Capa 1
│   ├── factory.py                        # Fábrica de adaptadores
│   └── implementation/
│       ├── camera.py                     # OpenCVCamera
│       ├── color_palette.py              # ColorPalette (2/4/8/16 colores)
│       ├── grid.py                       # Grid64Codec
│       ├── compressor.py                  # ZstdCompressor
│       ├── queue.py                       # FifoFrameQueue
│       ├── selector.py                   # AdapterSelector
│       ├── qr_adapter.py                  # QRLightAdapter
│       ├── ethernet_adapter.py            # EthernetAdapter (TCP)
│       ├── wifi_adapter.py                # WifiAdapter (UDP multicast)
│       └── display.py                     # OpenCVFrameRenderer
├── red/                                   # Capas 2/3 – remote-QR-net
│   ├── interfaces.py                     # IRoutingTable, INodeDirectory, etc.
│   ├── packet.py                         # Packet de Capa 3
│   └── implementation/
│       ├── routing.py                     # RoutingTable + CircuitTable
│       ├── directory.py                   # NodeDirectory
│       └── node.py                         # QRNetNode
├── capa7_anon/                             # Capa 7 – aplicación anónima
│   ├── app.py                            # AnonChatApp
│   ├── cli.py                             # Interfaz de línea de comandos
│   └── messages.py                        # PrivateMessage
├── tests/
│   ├── transmision/                       # Tests de Capa 1
│   ├── red/                               # Tests de Capa 2/3
│   └── capa7_anon/                         # Tests de Capa 7
├── docs/
│   └── rfc-qrnet.txt                       # RFC del protocolo (ASCII puro)
```

```
|   |─ diag.puml           # Diagrama UML PlantUML
|   |─ kick-off.md        # Documento de kick-off
|   └─ tools/
|       └─ find_camera.py  # Utilidad para encontrar ID de cámara
```

Flujo de trabajo Git

Se utilizó la estrategia de ramas por característica:

- **main** — código estable y revisado
- **feat/capa1** — desarrollo de la capa física
- **feat/capa2-3** — desarrollo de la capa de red
- **feat/capa7** — desarrollo de la aplicación
- **feat/rfc** — redacción del RFC
- **feat/adapters** — EthernetAdapter y WifiAdapter

Cada rama se integró a **main** mediante Pull Request con revisión del equipo.

3. Estructuras de Datos y Funciones

3.1 Capa 1 — Canal Óptico

HandshakeFrame (**transmision/frames.py**)

Dataclass que representa el primer frame de toda sesión QR-NET. Se usa para negociar capacidades antes de transmitir datos. Serializa a 52 bytes big-endian con checksum CRC-16.

Campo	Bits	Descripción
version	4	Versión del protocolo (0x1)
frame_type	4	SYN / SYN-ACK / ACK
magic	16	0x4E51 — identifica el protocolo
src_mac	48	MAC del emisor
dst_mac	48	MAC del receptor (broadcast = 0xFF*6)
color_depth	2	Paleta: 2/4/8/16 colores
grid_size	6	Tamaño de la grilla
interval_ms	16	Intervalo entre frames en ms
compression	4	Algoritmo de compresión
total_frames	32	Cantidad total de DataFrames
file_size	64	Tamaño del archivo original
file_hash	128	SHA-128 del contenido original

Campo	Bits	Descripción
<code>checksum</code>	16	CRC-16 de todos los bytes anteriores

Métodos clave:

- `to_bytes()` → `bytes`: serializa a wire format
- `from_bytes(data)` → `HandshakeFrame`: deserializa y verifica CRC-16
- `negotiate_with(remote)` → `HandshakeFrame`: calcula los parámetros acordados (mínimo de capacidades)

DataFrame (`transmission/frames.py`)

Dataclass que representa una unidad de datos, NACK o FIN. **No puede exceder 128 bytes en total.** Header fijo de 16 bytes + payload máximo de 110 bytes + CRC-16 de 2 bytes.

Campo	Bits	Descripción
<code>version</code>	4	Versión del protocolo
<code>frame_type</code>	4	DATA / NACK / FIN / FIN-ACK
<code>src_mac</code>	48	MAC del emisor
<code>dst_mac</code>	48	MAC del receptor
<code>seq_num</code>	16	Número de secuencia (0 = HandshakeFrame)
<code>payload_len</code>	8	Longitud del payload
<code>payload</code>	≤880	Datos (máx. 110 bytes)
<code>checksum</code>	16	CRC-16

Factories:

- `DataFrame.nack(src, dst, missing_seq)`: construye un NACK
- `DataFrame.fin(src, dst, seq)`: construye un FIN

Grid64Codec (`transmission/implementation/grid.py`)

Implementa la codificación y decodificación de una grilla de 64×64 módulos en una imagen BGR.

Grid 64×64 módulos:

SZ	SZ	SZ	...	SZ	SZ	SZ		← zona silencio (2 módulos)
SZ	FP7	FP7	...	FP7	.	.	SZ	
SZ	FP7	FP7	...	.	.	.	SZ	← finder patterns 7×7
SZ	.	.	...	.	.	.	SZ	
SZ	.	.	[CAL]	.	.	SZ		← calibration patch 4×4
SZ	FP7	FP7	...	.	FP7	SZ		

```
| SZ  SZ  SZ  ...  SZ  SZ  SZ  |
```

Métodos clave:

- `encode_grid(data, codec) → np.ndarray`: codifica bytes en imagen BGR
- `decode_grid(image, codec) → bytes`: decodifica imagen a bytes con alineación automática
- `_align_grid(image) → np.ndarray | None`: detecta y corrige perspectiva usando los finder patterns
- `_extract_cal_patch(aligned) → np.ndarray`: extrae el parche de calibración

ColorPalette (transmision/implementation/color_palette.py)

Implementa la codificación multicolor por módulo.

n_colors	bits/módulo	Paleta
2	1	Negro, Blanco
4	2	Negro, Blanco, Rojo, Cyan
8	3	+Verde, Magenta, Azul, Amarillo
16	4	Paleta HSV uniforme de 16 colores

- `encode(symbol) → np.ndarray`: símbolo entero → array RGB
- `decode(pixel) → int`: array RGB → símbolo más cercano (distancia euclidiana)
- `calibrate(cal_patch)`: ajusta los centroides de color con el parche de referencia

DispositivoLuzAdaptador (transmision/adapter.py)

Clase principal de Capa 1. Implementa `INetworkAdapter` para el canal óptico.

```
adaptador = DispositivoLuzAdaptador(
    mac=mac_bytes,
    camera=OpenCVCamera(device_id=700),
    color_codec=ColorPalette(n_colors=4),
    grid_codec=Grid64Codec(),
    compressor=ZstdCompressor(level=3),
    frame_queue=FifoFrameQueue(maxsize=32),
)
adaptador.start()          # abre cámara, lanza hilos PRODUCTOR y CONSUMIDOR
adaptador.do_handshake()   # negocia parámetros con el par
adaptador.send(b"datos")   # comprime, fragmenta y transmite en QR
data = adaptador.receive()
adaptador.stop()
```

Pipeline de hilos:

```
Hilo PRODUCTOR                                Hilo CONSUMIDOR
camera.capture() → change_detect → FIFO → decode_grid() → _rx_buffer
```

AdapterSelector (transmision/implementation/selector.py)

Selector de canal con política configurable. Soporta las políticas `PREFER_QR`, `PREFER_TCP`, `LOWEST_COST` y `FIRST_AVAILABLE`. Registra múltiples adaptadores y elige el mejor para cada destino.

EthernetAdapter (transmision/implementation/ethernet_adapter.py)

Implementa `INetworkAdapter` sobre TCP/IP. Usa un servidor TCP persistente para recepción y conexiones efímeras para envío. Framing con header de 4 bytes de longitud. Costo por defecto: 10.

WifiAdapter (transmision/implementation/wifi_adapter.py)

Implementa `INetworkAdapter` sobre UDP multicast IPv4. Grupo: `239.255.60.60:9001`. Incluye filtrado de eco por `SRC_MAC`. Costo por defecto: 20.

3.2 Capa 2/3 — remote-QR-net

Packet (red/packet.py)

PDU de Capa 3. Header de 46 bytes con CRC-32.

Campo	Bytes	Descripción
<code>version</code>	1	Versión del protocolo (0x1)
<code>packet_type</code>	1	DATA/HELLO/BYE/CIRCUIT_SETUP/CIRCUIT_TEARDOWN/ROUTE_UPDATE
<code>ttl</code>	1	Time To Live (default: 16)
<code>src_id</code>	16	NodeID efímero del origen (o <code>circuit_id</code> para paquetes anónimos)
<code>dst_id</code>	16	NodeID destino (0xFF×16 = broadcast)
<code>circuit_id</code>	4	ID de circuito virtual (0 = no anónimo)
<code>payload_len</code>	2	Longitud del payload
<code>payload</code>	variable	Datos
<code>checksum</code>	4	CRC-32

Factories: `Packet.hello()`, `Packet.bye()`, `Packet.circuit_setup()`, `Packet.circuit_teardown()`

RoutingTable (red/implementation/routing.py)

Tabla de ruteo distance-vector en memoria. Thread-safe mediante `threading.Lock`.

- `add_route(dst_id, next_hop_mac, cost)`: instala ruta si el costo es mejor o igual

- `lookup(dst_id) → RouteEntry | None`: busca la mejor ruta
- `purge_stale(max_age_s) → int`: elimina rutas obsoletas
- `all_routes() → list[RouteEntry]`: retorna todas las rutas activas

CircuitTable (red/implementation/routing.py)

Tabla de circuitos virtuales efimeros para ruteo anónimo. Cada entrada guarda (`prev_hop_mac`, `next_hop_mac`) para un `circuit_id`.

- `add_circuit(circuit_id, prev_hop_mac, next_hop_mac)`
- `lookup(circuit_id) → tuple | None`
- `remove_circuit(circuit_id)`

NodeDirectory (red/implementation/directory.py)

Registro en memoria de nodos conocidos en la mesh. Thread-safe.

- `register(node: NodeInfo)`: registra o actualiza (mantiene menor costo, actualiza `last_seen`)
- `lookup(node_id) → NodeInfo | None`
- `expire_stale(max_age_s) → int`: elimina nodos que no han enviado HELLO

QRNetNode (red/implementation/node.py)

Nodo de la mesh. Implementa `IQRNetNode`. Coordina ruteo, anonimato y descubrimiento.

```
node = QRNetNode(selector=adapter_selector)
node.join_mesh()                # lanza hilos RX y HELLO
cid = node.negotiate_circuit(dst_id) # negocia circuito anónimo
node.send_anonymous(b"mensaje", dst_id) # envío anónimo
msg = node.receive()             # lectura del buffer local
node.leave_mesh()                # envía BYE y detiene hilos
```

Características implementadas:

1. **Verificación de checksum obligatoria** en `route_packet()` — descarta paquetes corruptos
2. **ROUTE_UPDATE activo** — emite la tabla de ruteo tras HELLO, BYE y periódicamente
3. **Backoff exponencial binario (BEB)** — contención del medio con delay aleatorio

3.3 Capa 7 — Aplicación Anónima

PrivateMessage (capa7_anon/messages.py)

Dataclass inmutable que representa un mensaje entre nodos. Serializa a JSON compacto.

Campo	Tipo	Descripción
<code>sender_alias</code>	str	Seudónimo efímero del emisor

Campo	Tipo	Descripción
<code>recipient_id</code>	bytes	NodeID destino (16 bytes)
<code>body</code>	str	Contenido del mensaje
<code>kind</code>	str	<code>direct</code> / <code>post</code> / <code>reply</code>
<code>topic</code>	str	Tópico del post (opcional)
<code>message_id</code>	str	UUID aleatorio
<code>timestamp</code>	float	Unix timestamp

- `to_bytes()` → bytes: serializa a JSON + codifica para transmisión
- `from_bytes(data)` → `PrivateMessage`: deserializa y valida

AnonChatApp (`capa7_anon/app.py`)

Aplicación de microblogging anónimo sobre la mesh. Gestiona el ciclo de vida de mensajes.

```
app = AnonChatApp(node=qrnet_node)
app.start()
app.publish("Hola mesh", topic="general")
app.send_direct(b"...", "Mensaje privado")
msgs = app.inbox()
app.stop()
```

CLI (`capa7_anon/cli.py`)

Interfaz de línea de comandos interactiva. Comandos disponibles:

Comando	Descripción
<code>post <tópico> <mensaje></code>	Publica en un tópico de la mesh
<code>direct <node_id_hex> <mensaje></code>	Envía mensaje directo
<code>inbox</code>	Muestra mensajes recibidos
<code>peers</code>	Lista nodos conocidos
<code>quit</code>	Sale de la aplicación

3.4 Módulos Comunes

`checksum.py`

- `crc16(data: bytes) → int`: CRC-16/CCITT-FALSE
- `crc32(data: bytes) → int`: CRC-32 estándar
- `sha128(data: bytes) → bytes`: SHA-256 truncado a 16 bytes

exceptions.py

Jerarquía de excepciones del dominio:

```
QRNetError
├── AdapterError      # error en adaptador de red
│   ├── EthernetError # error TCP
│   └── WifiError     # error UDP multicast
├── ChecksumError    # CRC inválido
├── HandshakeError   # fallo en negociación de sesión
├── GridDecodeError  # imposible decodificar la grilla
└── RoutingError     # sin ruta al destino
```

4. Instrucciones para Ejecutar el Programa

Prerequisitos

```
# Clonar el repositorio
git clone <url-del-repositorio>
cd proyecto-qr

# Crear y activar el entorno virtual
python -m venv .venv
.venv\Scripts\activate      # Windows
source .venv/bin/activate   # Linux/macOS

# Instalar dependencias
pip install opencv-python numpy zstandard pillow screeninfo pytest cv2-enumerate-
cameras
```

4.1 Ejecutar los tests unitarios

```
# Todos los tests
python -m pytest tests/ -v

# Solo Capa 1
python -m pytest tests/transmision/test_transmision_layer.py -v

# Solo Capa 2/3
python -m pytest tests/red/test_red_layer.py -v

# Solo adaptadores Ethernet/WiFi
python -m pytest tests/transmision/test_adapters_ethernet_wifi.py -v

# Solo Capa 7
python -m pytest tests/capa7_anon/ -v
```

Resultado esperado: **178 tests pasando.**

4.2 Demo de integración en memoria (sin hardware)

Simula dos nodos QRNetNode comunicándose por canal en memoria:

```
python tests/integration_demo.py
```

Salida esperada:

```
[1] Intercambiando HELLOs...      OK
[2] Negociando circuito anonimo... OK
[3] Enviando mensaje anonimo...   OK
[4] Verificando anonimato...      OK
```

4.3 Encontrar el ID de cámara

```
python tools/find_camera.py
# Salida ejemplo:
# ID: 0      | Nombre: Integrated Webcam
# ID: 700    | Nombre: OBS Virtual Camera
```

4.4 Test con hardware real (cámara)

```
# Verificar que la cámara responde (cambiar 700 por el ID encontrado)
python -c "
import cv2, sys
cap = cv2.VideoCapture(700, cv2.CAP_DSHOW)
```

```
ret, frame = cap.read()
print('OK:', frame.shape if ret else 'sin frame')
cap.release()
"
```

Ejecutar el demo de hardware
python tests/transmision/test_qr_pipeline.py

4.5 Simulación de cámara con detección en tiempo real

```
python tests/transmision/camera_simulation.py
```

Controles durante la ejecución:

Tecla	Acción
Q / ESC	Salir
C	Cambiar color depth (2→4→8→16 colores)
+ / -	Aumentar/reducir intervalo entre frames
ESPACIO	Pausar/reanudar transmisión

4.6 Test de loopback óptico

Verifica la cadena completa encode → pantalla → screenshot → decode sin hardware adicional:

```
python tests/loopback_optico.py
```

4.7 Iniciar la aplicación de chat anónimo

```
# Nodo 1 (en una terminal)
python -m capa7_anon.cli

# Nodo 2 (en otra terminal o máquina)
python -m capa7_anon.cli
```

Comandos de ejemplo dentro de la CLI:

```
> peers                                # ver nodos conocidos
> post general "Hola desde QR-NET"
> inbox                                # ver mensajes recibidos
```

```
> direct aabbccdd... "Mensaje privado"
> quit
```

4.8 Probar con celular como cámara IP

1. Instalar **IP Webcam** (Android) desde Play Store y tocar *Start Server*
2. Anotar la URL que muestra la app (ej. <http://192.168.1.105:8080>)
3. Ejecutar:

```
python tests/transmision/test_qr_pipeline.py --url http://192.168.1.105:8080/video
```

5. Actividades Realizadas por Estudiante

Ion Dolanescu — Capa 1

Fecha	Actividad	Horas
07/04/2026	Diseño e implementación de Grid64Codec (encode/decode básico)	6
09/04/2026	Implementación de ColorPalette (2 y 4 colores, encode/decode)	2
11/04/2026	Integración de OpenCV para renderizado y captura de frames	5
14/04/2026	Pipeline FIFO productor-consumidor y FifoFrameQueue	3
16/04/2026	Implementación de HandshakeFrame con serialización y CRC-16	3
18/04/2026	Implementación de DataFrame y límite de 128 bytes	4
21/04/2026	Three-way handshake en DispositivoLuzAdaptador	4
23/04/2026	Soporte multicolor (8 y 16 colores), calibración de color	5
25/04/2026	Compresión Zstandard y ZstdCompressor	1
28/04/2026	send_file() con fragmentación y datacasting	2
30/04/2026	Fix de handshake responder (campos faltantes en SYN-ACK)	3
02/05/2026	Fix NACK con retransmisión desde caché y FIN/FIN-ACK	4
05/05/2026	Tests unitarios de Capa 1 (67 tests)	3
07/05/2026	Debugging con camera_simulation.py y corrección de doble alineación	6
Total		51 h

Ariana Jiménez — Capa 2/3, Adaptadores, RFC

Fecha	Actividad	Horas
08/04/2026	Diseño de interfaces abstractas de Capa 2/3 (IRoutingTable , etc.)	3

Fecha	Actividad	Horas
10/04/2026	Implementación de <code>RoutingTable</code> y <code>CircuitTable</code> (thread-safe)	3
12/04/2026	Implementación de <code>NodeDirectory</code> con expiración de entradas	3
15/04/2026	Diseño e implementación de <code>Packet</code> con CRC-32 y factories	4
17/04/2026	Implementación de <code>QRNetNode</code> (HELLO, BYE, ruteo básico)	4
20/04/2026	Circuitos virtuales anónimos en <code>QRNetNode.negotiate_circuit()</code>	4
22/04/2026	Tests unitarios de Capa 2/3 (75 tests)	3
24/04/2026	<code>EthernetAdapter</code> (TCP) con framing de longitud	3
26/04/2026	<code>WifiAdapter</code> (UDP multicast) con filtrado de eco	4
28/04/2026	Tests de <code>EthernetAdapter</code> y <code>WifiAdapter</code> (36 tests)	3
30/04/2026	Fix <code>SO_EXCLUSIVEADDRUSE</code> para Windows en <code>EthernetAdapter</code>	1
02/05/2026	ROUTE_UPDATE activo: propagación tras HELLO, BYE y periódica	2
04/05/2026	Backoff exponencial binario (BEB) en <code>_send_via_mac()</code>	2
07/05/2026	Fix checksum en <code>route_packet()</code> — descartar paquetes corruptos	1
10/05/2026	Redacción del RFC (secciones 1-7, diagramas ASCII)	3
13/05/2026	Completar RFC (secciones 8-13, estado de máquina, seguridad)	2
Total		45 h

Jose Mario Jiménez — Capa 7

Fecha	Actividad	Horas
14/04/2026	Diseño de <code>PrivateMessage</code> y tipos de mensaje (direct/post/reply)	4
16/04/2026	Serialización JSON de mensajes y validación de campos	3
18/04/2026	Implementación de <code>AnonChatApp</code> (ciclo de vida, inbox)	5
21/04/2026	Integración de <code>AnonChatApp</code> con <code>QRNetNode</code>	4
23/04/2026	Implementación de <code>CLI</code> con comandos post, direct, inbox, peers	5
25/04/2026	Tests de la interfaz anónima de Capa 7	4
28/04/2026	Tests de multicast en Capa 7	3
30/04/2026	Tests del nodo privado y la aplicación privada	3
03/05/2026	Corrección de integración con nueva API de <code>QRNetNode</code>	3
06/05/2026	Pruebas end-to-end entre dos instancias	4
Total		38 h

Total del equipo: 171 horas

6. Comentarios Finales

Estado del programa

Componente	Estado	Notas
Capa 1 — Grid64Codec	✓ Completo	Encode/decode verificado con 16 colores
Capa 1 — Handshake 3-way	✓ Completo	SYN/SYN-ACK/ACK con negociación de capacidades
Capa 1 — DataFrame 128 bytes	✓ Completo	MAX_PAYLOAD = 110 bytes, total $\leq$ 128 bytes
Capa 1 — NACK + retransmisión	✓ Completo	Cache de frames enviados, retransmisión selectiva
Capa 1 — FIN / FIN-ACK	✓ Completo	Espera FIN-ACK hasta 10 segundos
Capa 1 — EthernetAdapter	✓ Completo	TCP con framing de longitud
Capa 1 — WifiAdapter	✓ Completo	UDP multicast con filtrado de eco
Capa 2/3 — Ruteo dinámico	✓ Completo	Distance-vector con ROUTE_UPDATE activo
Capa 2/3 — Circuitos anónimos	✓ Completo	circuit_id efímero, SRC_ID anonimizado
Capa 2/3 — Directorio de nodos	✓ Completo	Con expiración por timeout
Capa 2/3 — BEB (contención)	✓ Completo	Backoff exponencial binario
Capa 7 — Chat anónimo	✓ Completo	Tipos direct/post/reply, CLI interactiva
Capa 7 — Clearnet (IRC/NNTP)	✖ Pendiente	No implementado en esta entrega
RFC	✓ Completo	1,781 líneas, ASCII puro, $\leq$ 72 chars/línea
Tests	✓ Completo	178 tests pasando

Problemas encontrados

- MSMF vs DirectShow en Windows:** La librería OpenCV en Windows usa por defecto el backend MSMF (Media Foundation) para acceder a la cámara, el cual se congela cuando otra aplicación (Teams, Zoom, navegador) ya está usando el dispositivo. Se resolvió usando `cv2.CAP_DSHOW` (DirectShow) en Windows, detectado automáticamente con `sys.platform == "win32"`.
- Doble alineación en `camera_simulation.py`:** El hilo consumidor llamaba `_debug_align()` para obtener la imagen ya alineada y luego la pasaba a `decode_grid()`, que internamente llama `_align_grid()` de nuevo. La imagen ya alineada ocupa el frame completo sin márgenes, por lo que el detector de contornos solo encontraba los tres finder patterns (cada uno con ~1% del área) en lugar del borde exterior del QR. Se corrigió pasando el frame original directamente a `decode_grid()`.
- SO_REUSEADDR en Windows:** En Windows, esta opción de socket permite que dos procesos simultáneos se bindeen al mismo puerto, comportamiento opuesto al de Linux donde solo permite re-bind tras `TIME_WAIT`. Se corrigió usando `SO_EXCLUSIVEADDRUSE` en Windows.

4. **send() retorna False en el demo de hardware:** `is_available()` en `DispositivoLuzAdaptador` requiere que `_negotiated` no sea `None`, lo cual solo ocurre tras un handshake completo con un par real. En el demo de hardware se inyecta un `HandshakeFrame` simulado para poder probar `send()` en ausencia de un segundo nodo.
5. **factory.py con parámetros inexistentes:** La fábrica llamaba a `QRLightAdapter` pasando `frame_queue` y `renderer`, que no existían en el constructor. Se corrigió eliminando esos parámetros de la llamada.

Limitaciones conocidas

- La Capa 7 pública (IRC, NNTP, bot de bridge) no fue implementada en esta entrega.
- El canal de Video Compartido (Capa 1) está definido en la arquitectura como `AdapterType.VIDEO_STREAM` pero no tiene una implementación concreta.
- La tasa de transferencia del canal óptico depende fuertemente de las condiciones de iluminación y la distancia entre la pantalla y la cámara. A 1 metro con buena iluminación se alcanzan ~19 KB/s sin compresión y hasta ~65 KB/s equivalentes con texto y zstd nivel 3.
- El sistema no implementa cifrado de capa 7; la confidencialidad del contenido depende del entorno físico.

7. Conclusiones y Recomendaciones

Conclusiones

1. **La luz visible es un medio de comunicación viable** para enlazar dos nodos en distancias cortas (~1-3 metros con hardware de consumo). La codificación multicolor aumenta la densidad de información en hasta 4× respecto al QR clásico en blanco y negro.
2. **La arquitectura de interfaces abstractas** (`INetworkAdapter`, `IGridCodec`, `IColorCodec`) fue fundamental para desarrollar cada capa de forma independiente y probar con mocks sin requerir hardware real. Los 178 tests unitarios verifican la lógica del protocolo sin necesitar cámara ni red.
3. **El modelo OSI se implementó de forma coherente:** cada capa expone una interfaz bien definida a la capa superior y tiene sus propias estructuras de datos (`HandshakeFrame/DataFrame` para Capa 2, `Packet` para Capa 3, `PrivateMessage` para Capa 7). Esto facilitó el desarrollo paralelo por los tres integrantes.
4. **El protocolo distance-vector con ROUTE_UPDATE activo** converge rápidamente cuando un nodo entra o sale de la mesh. El backoff exponencial binario previene colisiones en el canal óptico compartido, aunque para una mesh de más de 2 nodos simultáneos se requeriría un mecanismo más sofisticado.
5. **El anonimato por circuitos virtuales** oculta efectivamente el Node ID del originador en los paquetes en tránsito. Sin embargo, para anonimato completo frente a adversarios globales que observan todos los enlaces, sería necesario añadir padding y timing aleatorio, al estilo de las redes de mezcla (mix networks).

Recomendaciones

1. **Cifrado de extremo a extremo:** Para uso real, los mensajes de Capa 7 deberían cifrarse (AES-GCM o ChaCha20-Poly1305) antes de enviarse por la mesh. El protocolo ya prevé versiones futuras mediante el campo **version** en todos los frames y paquetes.
 2. **ECC (Error Correction Code):** El campo **ECC** en el HandshakeFrame está definido pero no implementado. Añadir Reed-Solomon o Turbo Codes a la grilla aumentaría significativamente la robustez del canal óptico ante iluminación adversa.
 3. **VideoStreamAdapter:** El cuarto canal físico definido (**VIDEO_STREAM**) permitiría operar QR-NET a través de videollamadas (Zoom, Meet, Teams), lo que eliminaría la limitación de distancia física del canal óptico directo.
 4. **Implementar la Capa 7 pública:** El servidor IRC con bot de bridge y publicación NNTP completaría la arquitectura del proyecto y permitiría que mensajes marcados como públicos sean accesibles desde Internet convencional.
 5. **Optimizar el selector de canal:** La política **LOWEST_COST** ya prioriza Ethernet sobre WiFi sobre QR, pero un mecanismo de failover automático (por ejemplo, cambiar a QR cuando TCP falla) haría el sistema más robusto para escenarios de censura de red.
-

8. Bibliografía

1. Fielding, R., et al. "Hypertext Transfer Protocol — HTTP/1.1", RFC 2616, IETF, Junio 1999. Disponible en: <https://www.rfc-editor.org/info/rfc2616>
2. ISO/IEC 18004:2015. "Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification", Tercera edición, 2015.
3. Bradner, S. "Key words for use in RFCs to Indicate Requirement Levels", RFC 2119, IETF, Marzo 1997. <https://www.rfc-editor.org/info/rfc2119>
4. Meyer, D. "Administratively Scoped IP Multicast", RFC 2365, IETF, Julio 1998. <https://www.rfc-editor.org/info/rfc2365>
5. Hedrick, C. "Routing Information Protocol", RFC 1058, IETF, Junio 1988. <https://www.rfc-editor.org/info/rfc1058>
6. Collet, Y. y Kucherawy, M. "Zstandard Compression and the 'application/zstd' Media Type", RFC 8878, IETF, Febrero 2021. <https://www.rfc-editor.org/info/rfc8878>
7. IEEE Std 802.3-2022. "IEEE Standard for Ethernet". IEEE, 2022.
8. Bradski, G. "The OpenCV Library". Dr. Dobb's Journal of Software Tools, 2000. <https://opencv.org>
9. Dingledine, R., Mathewson, N., y Syverson, P. "Tor: The Second-Generation Onion Router". USENIX Security Symposium, 2004.
10. Tanenbaum, A. S. y Wetherall, D. J. "Computer Networks", 5ta edición. Pearson, 2011.
11. Documentación oficial de Python. "threading — Thread-based parallelism". <https://docs.python.org/3/library/threading.html>

12. Documentación oficial de Python. "socket — Low-level networking interface".
<https://docs.python.org/3/library/socket.html>